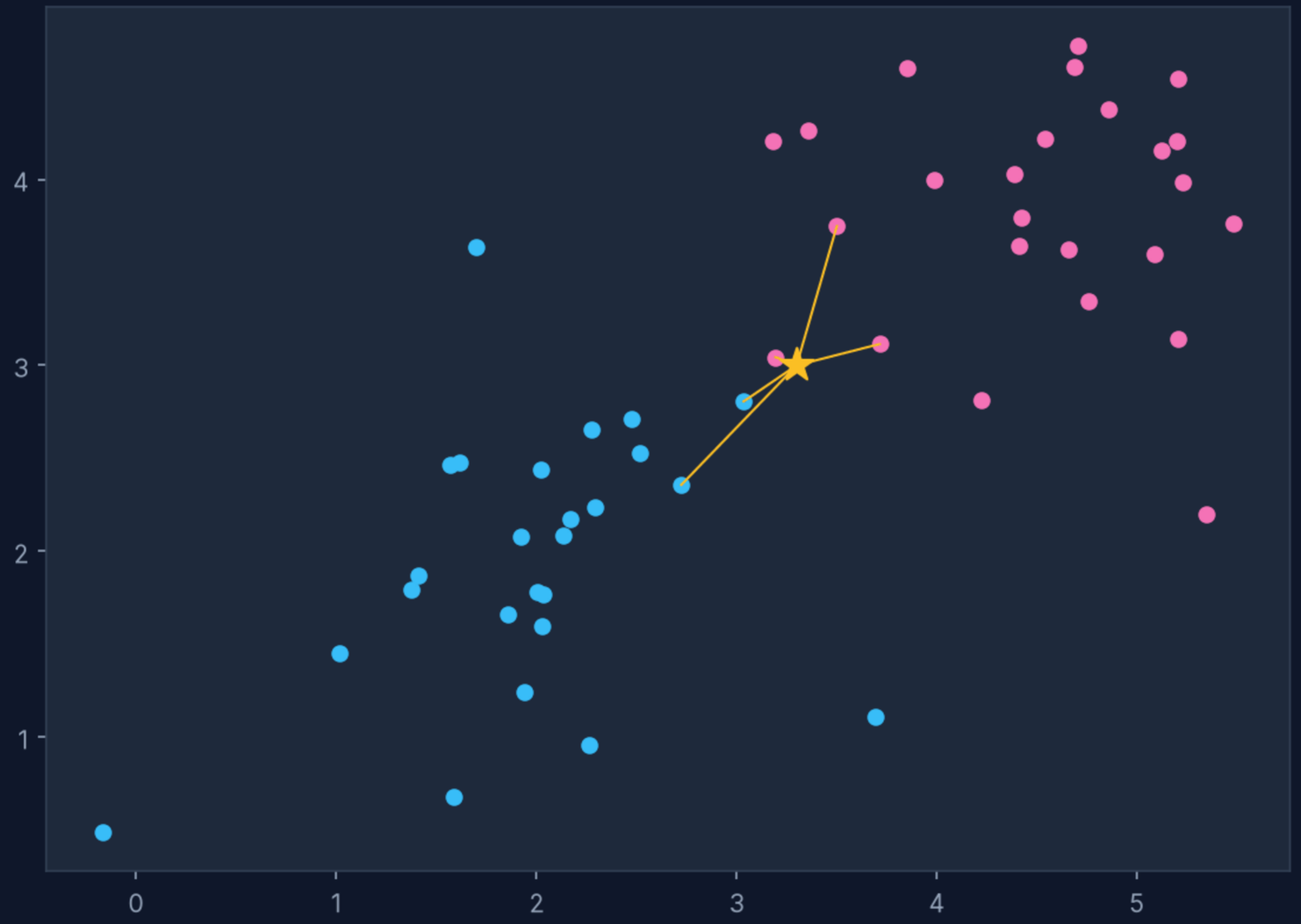
k-Nearest Neighbours

Predict by asking your neighbours

The problem

- Classify or regress a new point
- No assumptions about the shape of the boundary
- Similar inputs should have similar outputs

Intuition



Find the k closest points

Classification: majority vote

Regression: average value

Distance

$$d(x, x') = \sqrt{\sum_j (x_j - x'_j)^2}$$

- Scale features first, or one feature dominates
- Odd k avoids ties in binary problems
- Small k: noisy; large k: over-smoothed

Implementation

```
def predict(self, X):
    X = np.asarray(X, float)
    d = ((X[:, None, :] - self.X[None, :, :]) ** 2).sum(-1)
    nn = np.argsort(d, axis=1)[:, :self.k]
    labels = self.y[nn]
    if self.task == "regression":
        return labels.mean(axis=1)
    out = []
    for row in labels:
        vals, counts = np.unique(row, return_counts=True)
        out.append(vals[np.argmax(counts)])
    return np.array(out)
```

Complexity & trade-offs

- Training O(1); prediction O(n·d) per query
- Memory: stores the whole dataset
- Curse of dimensionality: distances lose meaning
- Speed-ups: KD-trees, ball trees, approximate NN

Recap

- Lazy learner: store, then vote or average
- Scale features; tune k
- Cheap to train, expensive to query
- Great baseline for low-dimensional data